2 POINT SOFT

GAMEPAD VIBRATION SYSTEM

Manual completo de usuario y referencia práctica



Motor	Unity Input System
API	C#, componentes, perfiles y UnityEvents
Versión del manual	1.0 — Español

Rumble estándar para gamepads mediante Unity Input System. Incluye One Shot, Pulsed, Continuous, mezcla por dispositivo, targeting local, perfiles reutilizables y detención selectiva.

Contenido

1. Introducción 4

2. Requisitos y compatibilidad 4

3. Instalación 4

Prueba rápida . . . 5

4. Conceptos fundamentales 5

Low Frequency . . . 5

High Frequency . . . 5

Duration . . . 5

Pulses e Interval . . . 5

Fade Out . . . 5

Global Intensity . . . 5

5. Modos de reproducción 5

Regla importante para Continuous . . . 6

6. Flujo sin código: componente GamepadVibration 6

Campos principales . . . 6

Métodos públicos . . . 6

Ejemplo con UnityEvent . . . 6

7. GamepadVibrationTrigger 7

Continuous Hazard sin código . . . 7

8. Presets integrados 7

9. GamepadVibrationSettings 8

10. GamepadVibrationManager 8

API histórica rápida . . . 8

Reproducir settings o presets . . . 8

Stop All . . . 8

Mezcla . . . 8

11. GamepadVibrationHandle y ownership 9

Ejemplo Continuous completo . . . 9

12. Perfiles personalizados 9

Profile Manager . . . 9
Usar un perfil desde C# . . . 10
Continuous mediante perfil . . . 10

13. API tipada generada opcional 10

14. Targeting y multiplayer local 11

Current Gamepad . . . 11
Gamepad explícito . . . 11
PlayerInput . . . 11
Recomendación . . . 11

15. Ejemplos prácticos 11

Disparo y retroceso . . . 11
Explosión con preset modificado . . . 11
Daño por peligro continuo . . . 12
UI mediante componente . . . 12

16. Escenas de demostración 12

GamepadVibrationDemo . . . 12
GamepadVibrationLocalMultiplayerDemo . . . 12
GamepadVibrationProfilesDemo . . . 12

17. Desconexiones y parada segura 12

18. Solución de problemas 13

No hay vibración . . . 13
Vibra el jugador equivocado . . . 13
Continuous no se detiene . . . 13
Falta un miembro generado . . . 13
DualSense . . . 13
La vibración continúa al cambiar de objeto . . . 13

19. Referencia rápida de tipos importantes 14

20. Límites del producto 14

21. Lista de comprobación para integración 14

Flujos visuales del producto 15

Editor: 2 Point Soft **Namespace principal:** `TwoPointSoft.GamepadVibration` **Tecnología:** Unity Input System
Documento: Manual de usuario y referencia práctica

> Gamepad Vibration System proporciona rumble estándar de motores de baja y alta frecuencia. No incluye adaptive triggers ni hápticos avanzados específicos de una plataforma.

1. Introducción

Gamepad Vibration System es una solución independiente para añadir vibración a juegos de Unity mediante `UnityEngine.InputSystem`. Puede utilizarse sin escribir código, desde UnityEvents, mediante componentes reutilizables, con perfiles `ScriptableObject` o directamente desde C#.

Funciones principales:

- Modos **One Shot**, **Pulsed** y **Continuous**.
- Control independiente de motores de baja y alta frecuencia.
- Presets integrados y perfiles personalizados reutilizables.
- API directa C# y métodos compatibles con UnityEvents.
- Mezcla de solicitudes simultáneas por dispositivo.
- Handles para detener únicamente el efecto que los creó.
- Targeting por `Gamepad` o `PlayerInput`.
- Limpieza ante desconexiones.
- Uso de tiempo no escalado.
- Profile Manager y API tipada opcional.
- Tres escenas de demostración.

El Manager se crea automáticamente cuando se solicita la primera vibración. No es necesario añadirlo manualmente a una escena.

2. Requisitos y compatibilidad

Elemento	Requisito o alcance
Unity	Validado con Unity 6000.3.15f1
Input	Unity Input System 1.19.0
Dispositivo	Gamepad reconocido por `UnityEngine.InputSystem.Gamepad`
Demo UI	Unity UGUI
Render pipeline	Runtime independiente de Built-in, URP y HDRP
DualSense	Rumble estándar validado por USB en Windows

El runtime no depende de Game Creator, TextMesh Pro ni paquetes de terceros. El comportamiento Bluetooth y la compatibilidad de cada dispositivo dependen de Unity Input System y de la plataforma.

3. Instalación

1. Importe la carpeta `Assets/2PointSoft/GamepadVibrationSystem` en el proyecto.
2. Abra **Window > Package Manager**.

3. Confirme que **Input System** está instalado.
4. Si Unity lo solicita, active el nuevo Input System y reinicie el Editor.
5. Conecte un gamepad compatible.
6. Abra `Demo/Scenes/GamepadVibrationDemo.unity` y entre en Play Mode.

Para la UI de las demos, confirme también la disponibilidad de Unity UGUI.

Prueba rápida

Añada el componente desde:

```
Add Component > 2 Point Soft > Gamepad Vibration
```

Configure una vibración, conecte `Play()` a un botón o evento y entre en Play Mode.

4. Conceptos fundamentales

Low Frequency

Controla el motor de vibración grave. Es útil para explosiones, maquinaria, golpes pesados, motores y eventos con sensación de peso.

High Frequency

Controla el motor más rápido. Es útil para disparos, impactos ligeros, UI, alertas y texturas rápidas.

Ambos valores utilizan el rango `0` a `1`.

Duration

Duración, en segundos, de cada pulso. El sistema utiliza tiempo no escalado, por lo que sigue funcionando aunque `Time.timeScale` cambie.

Pulses e Interval

`Pulses` define cuántas veces se repite el efecto. `Interval` establece la pausa entre pulsos.

Fade Out

Reduce gradualmente las intensidades durante cada pulso.

Global Intensity

Multiplicador final aplicado a ambos motores. Permite conservar la relación entre Low y High mientras se ajusta la fuerza general.

5. Modos de reproducción

Modo	Comportamiento	Uso típico
One Shot	Ejecuta un único pulso finito	Golpe, salto, selección UI
Pulsed	Repite varios pulsos con intervalos	Alerta, latido, ráfaga
Continuous	Vibra indefinidamente hasta llamar a Stop	Motor, peligro, maquinaria

Regla importante para Continuous

Continuous no utiliza una duración final. Debe detenerse mediante el `GamepadVibrationHandle` devuelto, el método `Stop()` del componente propietario o una operación de Stop dirigida al dispositivo.

6. Flujo sin código: componente GamepadVibration

`GamepadVibration` es el componente principal para diseñadores.

Campos principales

Campo	Descripción
Source	Usa valores/preset integrados o un `GamepadVibrationProfile`
Preset	Selecciona un preset protegido o valores Custom
Playback Mode	One Shot, Pulsed o Continuous
Target Mode	`CurrentGamepad` o `PlayerInput`
Player Input	PlayerInput opcional para multiplayer local
Low / High Frequency	Intensidad de cada motor
Duration	Duración de cada pulso
Pulses	Cantidad de pulsos
Interval	Pausa entre pulsos
Fade Out	Disminución gradual
Global Intensity	Multiplicador final

Métodos públicos

```
public void Play();
public void Stop();
public void ApplyPreset(GamepadVibrationPreset newPreset);
```

`Stop()` detiene únicamente solicitudes creadas por ese componente. Al deshabilitarse o destruirse, el componente limpia sus propios efectos sin detener vibraciones ajenas.

Ejemplo con UnityEvent

1. Añada `GamepadVibration` a un GameObject.
2. Configure el preset o los valores.
3. En un `Button.onClick`, arrastre el GameObject.
4. Seleccione `GamepadVibration > Play()`.
5. Para detener Continuous, conecte otro evento a `GamepadVibration > Stop()`.

Esto también funciona con Animation Events, eventos de gameplay propios y UnityEvents serializados.

7. GamepadVibrationTrigger

`GamepadVibrationTrigger` permite reproducir y detener efectos mediante colliders 3D.

Campos:

- **Vibration:** componente `GamepadVibration` que será controlado.
- **Trigger Event:** reproduce en `OnEnter` o `OnExit`.
- **Required Tag:** limita quién puede activar el trigger; vacío acepta cualquier tag.
- **Trigger Once:** permite una sola activación.
- **Stop On Opposite Event:** detiene al ocurrir el evento contrario; recomendado para Continuous.

Continuous Hazard sin código

1. Añada un collider y active **Is Trigger**.
2. Añada `GamepadVibration` y elija **Continuous**.
3. Añada `GamepadVibrationTrigger`.
4. Configure **Trigger Event = OnEnter**.
5. Active **Stop On Opposite Event**.
6. Use el tag `Player` en el objeto que entra.

El efecto comienza al entrar y se detiene al salir.

Para permitir otra activación cuando `Trigger Once` está activo:

```
trigger.ResetTrigger();
```

8. Presets integrados

El paquete incluye:

- `LightHit`
- `HeavyHit`
- `Explosion`
- `Heartbeat`
- `Alert`
- `Hazard`
- `UISelect`
- `UIConfirm`

Ejemplo directo:

```
using TwoPointSoft.GamepadVibration;

GamepadVibrationPresets.HeavyHit.Play();
```

Crear una variante sin modificar el preset:

```
GamepadVibrationSettings custom = GamepadVibrationPresets.HeavyHit.With(
    duration: 0.35f,
    globalIntensity: 0.75f,
    fadeOut: true);

GamepadVibrationHandle handle = custom.Play();
```

Los presets integrados son valores protegidos. Para editarlos de forma persistente, cópielos como perfil personalizado desde el Profile Manager.

9. GamepadVibrationSettings

`GamepadVibrationSettings` es una estructura inmutable con valores validados.

```
GamepadVibrationSettings settings = new GamepadVibrationSettings(
    lowFrequency: 0.65f,
    highFrequency: 0.90f,
    duration: 0.18f,
    pulses: 3,
    interval: 0.08f,
    fadeOut: true,
    globalIntensity: 0.85f);

GamepadVibrationHandle handle = settings.Play();
```

Los valores de intensidad se limitan automáticamente entre `0` y `1`; duración e intervalo no pueden ser negativos; pulsos nunca es menor que uno.

Métodos principales:

```
settings.Play();
settings.Play(gamepad);
settings.Play(playerInput);
settings.With(...);
```

10. GamepadVibrationManager

`GamepadVibrationManager` es el núcleo estático del sistema. Mantiene solicitudes, mezcla efectos, actualiza motores y limpia dispositivos desconectados.

API histórica rápida

```
GamepadVibrationManager.Play(
    low: 0.5f,
    high: 0.8f,
    duration: 0.2f,
    fadeOut: true);
```

Sin target explícito se usa `Gamepad.current`.

Reproducir settings o presets

```
GamepadVibrationHandle hit =
    GamepadVibrationManager.Play(GamepadVibrationPresets.HeavyHit);

GamepadVibrationHandle alert =
    GamepadVibrationManager.PlayPreset(GamepadVibrationPreset.Alert);
```

Stop All

```
GamepadVibrationManager.StopAll();
GamepadVibrationManager.StopAll(gamepad);
GamepadVibrationManager.StopAll(playerInput);
```

Use `StopAll()` como acción global o de emergencia. Para gameplay normal con efectos superpuestos, prefiera detener el handle o componente propietario.

Mezcla

Las solicitudes simultáneas se suman por dispositivo y se limitan al rango válido antes de enviarse al gamepad. Un efecto no reemplaza automáticamente a los demás. Cada gamepad mantiene una mezcla independiente.

11. GamepadVibrationHandle y ownership

Un `GamepadVibrationHandle` representa una secuencia iniciada por script.

```
GamepadVibrationHandle handle = GamepadVibrationPresets.Alert.Play();

// Más tarde:
handle.Stop();
```

Detener un handle no cancela solicitudes ajenas. Esta propiedad es especialmente importante para Continuous y para sistemas donde UI, combate, vehículos y ambiente pueden vibrar al mismo tiempo.

Ejemplo Continuous completo

```
using UnityEngine;
using UnityEngine.InputSystem;
using TwoPointSoft.GamepadVibration;

public sealed class EngineRumbleExample : MonoBehaviour
{
    private GamepadVibrationHandle engineHandle;

    public void StartEngine()
    {
        Gamepad target = Gamepad.current;
        if (target == null) return;

        engineHandle?.Stop();
        engineHandle = GamepadVibrationManager.PlayContinuous(
            low: 0.45f,
            high: 0.15f,
            target: target);
    }

    public void StopEngine()
    {
        engineHandle?.Stop();
        engineHandle = null;
    }

    private void OnDisable()
    {
        StopEngine();
    }
}
```

12. Perfiles personalizados

`GamepadVibrationProfile` es un `ScriptableObject` reutilizable. Permite centralizar una sensación y usarla desde múltiples escenas y objetos.

Crear desde el menú de assets:

```
Create > 2 Point Soft > Gamepad Vibration Profile
```

También puede utilizar:

```
Tools > 2 Point Soft > Vibration Profile Manager
```

Profile Manager

El Profile Manager permite:

- buscar y filtrar perfiles;
- inspeccionar presets integrados protegidos;
- crear desde cero;

- copiar un built-in;
- copiar un perfil existente;
- editar mediante working copy;
- guardar cambios explícitamente;
- restaurar cambios no guardados;
- previsualizar en Play Mode;
- duplicar y eliminar perfiles custom;
- generar la API C# opcional.

Save Profile guarda el working copy. **Reset Unsaved Changes** restaura el último Save explícito.

Usar un perfil desde C#

```
using UnityEngine;
using TwoPointSoft.GamepadVibration;

public sealed class SwordExample : MonoBehaviour
{
    [SerializeField] private GamepadVibrationProfile swordAttack;

    public void Attack()
    {
        GamepadVibrationHandle handle = swordAttack?.Play();
    }
}
```

Continuous mediante perfil

```
private GamepadVibrationHandle idleHandle;

public void StartIdle()
{
    idleHandle?.Stop();
    idleHandle = engineIdleProfile.Play();
}

public void StopIdle()
{
    idleHandle?.Stop();
    idleHandle = null;
}
```

13. API tipada generada opcional

La generación crea:

```
Assets/GamepadVibrationGenerated/GamepadVibrationCustom.cs
```

No es obligatoria. Los perfiles, componentes y demos funcionan sin ese archivo.

Ejemplo:

```
using TwoPointSoft.GamepadVibration;
using TwoPointSoft.GamepadVibration.Generated;

GamepadVibrationCustom.SwordAttack.Play();
GamepadVibrationCustom.ShotgunRecoil.Play(playerInput);

GamepadVibrationHandle idle =
    GamepadVibrationCustom.EngineIdle.Play(gamepad);
idle.Stop();
```

Después de crear, renombrar o eliminar perfiles, use **Generate API** o **Update API** y espere la recompilación de Unity.

14. Targeting y multiplayer local

Current Gamepad

Las APIs sin target usan `Gamepad.current`. Es apropiado para single-player y herramientas rápidas.

```
GamepadVibrationPresets.LightHit.Play();
```

Gamepad explícito

```
Gamepad target = Gamepad.current;
if (target != null)
{
    GamepadVibrationPresets.Explosion.Play(target);
}
```

PlayerInput

```
using UnityEngine.InputSystem;

[SerializeField] private PlayerInput playerInput;

public void DamagePlayer()
{
    GamepadVibrationPresets.HeavyHit.Play(playerInput);
}
```

El sistema selecciona el primer `Gamepad` emparejado con ese `PlayerInput`. Un target explícito inválido no hace fallback a otro jugador.

Recomendación

No use `Gamepad.all[playerIndex]` como identidad permanente. El orden puede cambiar al conectar o desconectar dispositivos. Guarde el `Gamepad` que se unió o utilice el `PlayerInput` del jugador.

15. Ejemplos prácticos

Disparo y retroceso

```
public void Fire()
{
    GamepadVibrationSettings recoil = new GamepadVibrationSettings(
        lowFrequency: 0.35f,
        highFrequency: 0.95f,
        duration: 0.08f,
        pulses: 1,
        fadeOut: true);

    recoil.Play();
}
```

Explosión con preset modificado

```
public void Explode(Gamepad target)
{
    GamepadVibrationSettings explosion =
        GamepadVibrationPresets.Explosion.With(globalIntensity: 0.65f);

    explosion.Play(target);
}
```

Daño por peligro continuo

```
private GamepadVibrationHandle hazardHandle;

public void EnterHazard(Gamepad target)
{
    hazardHandle?.Stop();
    hazardHandle = GamepadVibrationManager.PlayContinuous(0.55f, 0.35f, target);
}

public void ExitHazard()
{
    hazardHandle?.Stop();
    hazardHandle = null;
}
```

UI mediante componente

Configure `UISelect` en un componente y conecte `EventTrigger.PointerEnter` o su sistema de navegación al método `Play()`. Para confirmar, use otro componente con `UIConfirm`.

16. Escenas de demostración

GamepadVibrationDemo

```
Demo/Scenes/GamepadVibrationDemo.unity
```

Incluye componentes, UnityEvents, presets, API directa, triggers, disparo, impacto, efectos pulsados y maquinaria Continuous.

GamepadVibrationLocalMultiplayerDemo

```
Demo/Scenes/GamepadVibrationLocalMultiplayerDemo.unity
```

Permite conectar hasta cuatro mandos. Cada slot guarda el `Gamepad` exacto que se unió. Los slots desconectados se liberan para aceptar un reemplazo.

GamepadVibrationProfilesDemo

```
Demo/Scenes/GamepadVibrationProfilesDemo.unity
```

Ejemplos incluidos:

1. **Sword Attack** — One Shot.
2. **Shotgun Recoil** — Pulsed.
3. **Engine Idle** — Continuous con Start y Stop independientes.

17. Desconexiones y parada segura

El Manager elimina solicitudes asociadas con dispositivos desconectados. Los targets explícitos inválidos no redirigen vibración a otro mando.

Cuando termina la última solicitud, se envía una confirmación de parada durante una ventana breve antes de liberar el dispositivo. Esto mejora la fiabilidad del rumble estándar HID sin mantener ownership indefinidamente.

Buenas prácticas:

- Guarde y detenga handles Continuous.
- Detenga efectos propios en `OnDisable`.
- Use Stop dirigido en multiplayer.

- Evite `StopAll()` para cancelar un único efecto.
- No conserve índices de `Gamepad.all` como identidad de jugador.

18. Solución de problemas

No hay vibración

- Confirme que Input System está instalado y activo.
- Conecte el mando y pulse un botón para establecer `Gamepad.current`.
- Compruebe que el dispositivo ofrece rumble estándar mediante Unity Input System.
- Revise que Low, High y Global Intensity sean mayores que cero.
- Entre en Play Mode antes de usar Preview.

Vibra el jugador equivocado

Use `Play(playerInput)`, un `Gamepad` explícito o Target Mode `PlayerInput`. No identifique jugadores con índices cambiantes.

Continuous no se detiene

Conserve el handle y llame `handle.Stop()`, o use `Stop()` en el componente que inició el efecto.

Falta un miembro generado

Abra el Profile Manager, pulse **Update API** o **Generate API** y espere que Unity termine de compilar.

DualSense

El rumble estándar fue validado por USB en Windows. Bluetooth depende del soporte de plataforma y Unity Input System. Adaptive triggers y hápticos avanzados no forman parte del producto.

La vibración continúa al cambiar de objeto

Compruebe que el propietario detenga su handle en `OnDisable`, `OnDestroy`, al salir del estado o al salir del trigger. Para triggers Continuous active **Stop On Opposite Event**.

19. Referencia rápida de tipos importantes

Tipo	Responsabilidad
`GamepadVibrationManager`	Núcleo, mezcla, targeting, Stop y desconexiones
`GamepadVibration`	Componente reutilizable y compatible con UnityEvents
`GamepadVibrationTrigger`	Reproduce/detiene mediante trigger 3D
`GamepadVibrationSettings`	Valores inmutables y validados
`GamepadVibrationPresets`	Presets integrados
`GamepadVibrationProfile`	Perfil custom reutilizable como asset
`GamepadVibrationHandle`	Ownership y detención selectiva
`GamepadVibrationGeneratedProfile`	Representación usada por la API generada
`GamepadVibrationSource`	Selecciona valores/preset o perfil
`GamepadVibrationPlaybackMode`	OneShot, Pulsed o Continuous
`GamepadVibrationTargetMode`	CurrentGamepad o PlayerInput

20. Límites del producto

Gamepad Vibration System ofrece rumble estándar de dos motores mediante Unity Input System. No incluye:

- adaptive triggers;
- hápticos avanzados específicos de PS5;
- networking;
- emparejamiento de jugadores por red;
- soporte garantizado para dispositivos que Unity no reconoce como `Gamepad`;
- una dependencia obligatoria de URP o HDRP.

La compatibilidad final depende del gamepad, conexión, sistema operativo, backend de entrada y versión de Unity Input System.

21. Lista de comprobación para integración

- Input System instalado y activo.
- Gamepad reconocido por Unity.
- Elegido el modo correcto.
- Continuous conserva su handle o componente propietario.
- Multiplayer usa `PlayerInput` o referencias `Gamepad` estables.
- UnityEvents conectados a `Play()` y `Stop()`.
- Perfiles guardados explícitamente.
- API generada actualizada solo si el proyecto la utiliza.
- Demos probadas con el hardware objetivo.

Flujos visuales del producto

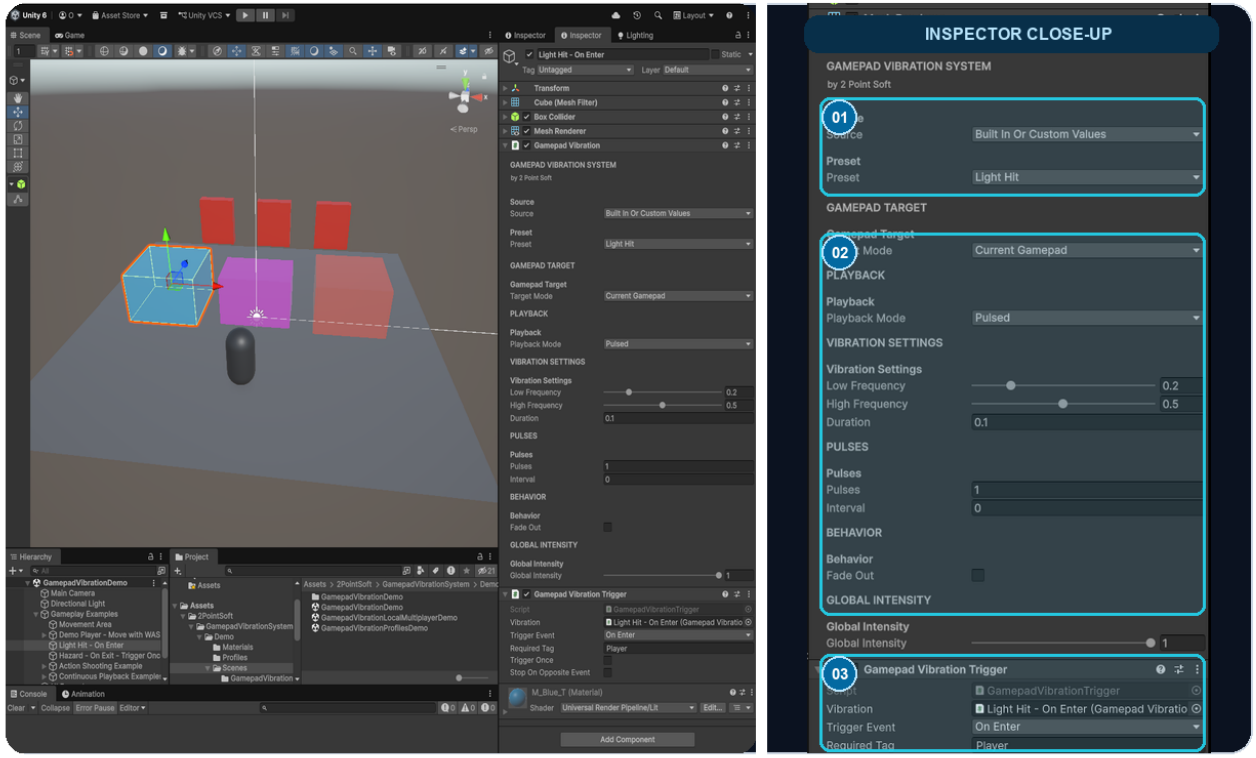
Estas láminas resumen los principales flujos del sistema. Se presentan en formato horizontal para conservar la legibilidad de la interfaz y los diagramas.

INSPECTOR WORKFLOW

SET UP VIBRATION WITHOUT CODE

Choose an effect, adjust its response, and trigger it directly from a GameObject.

GAMEPAD VIBRATION SYSTEM



NO-CODE SETUP

01 Choose the source
Use a built-in preset, custom values, or a reusable vibration profile.

02 Shape the response
Select the target and playback mode, then tune both motors and timing.

03 Trigger from gameplay
Play on Enter or Exit, filter by tag, and optionally stop on the opposite event.

DESIGNER-FRIENDLY
Configure effects in the Inspector and connect them to reusable gameplay triggers.

COMPONENT WORKFLOW PRESETS, CUSTOM VALUES, AND PROFILES

UNITY INPUT SYSTEM

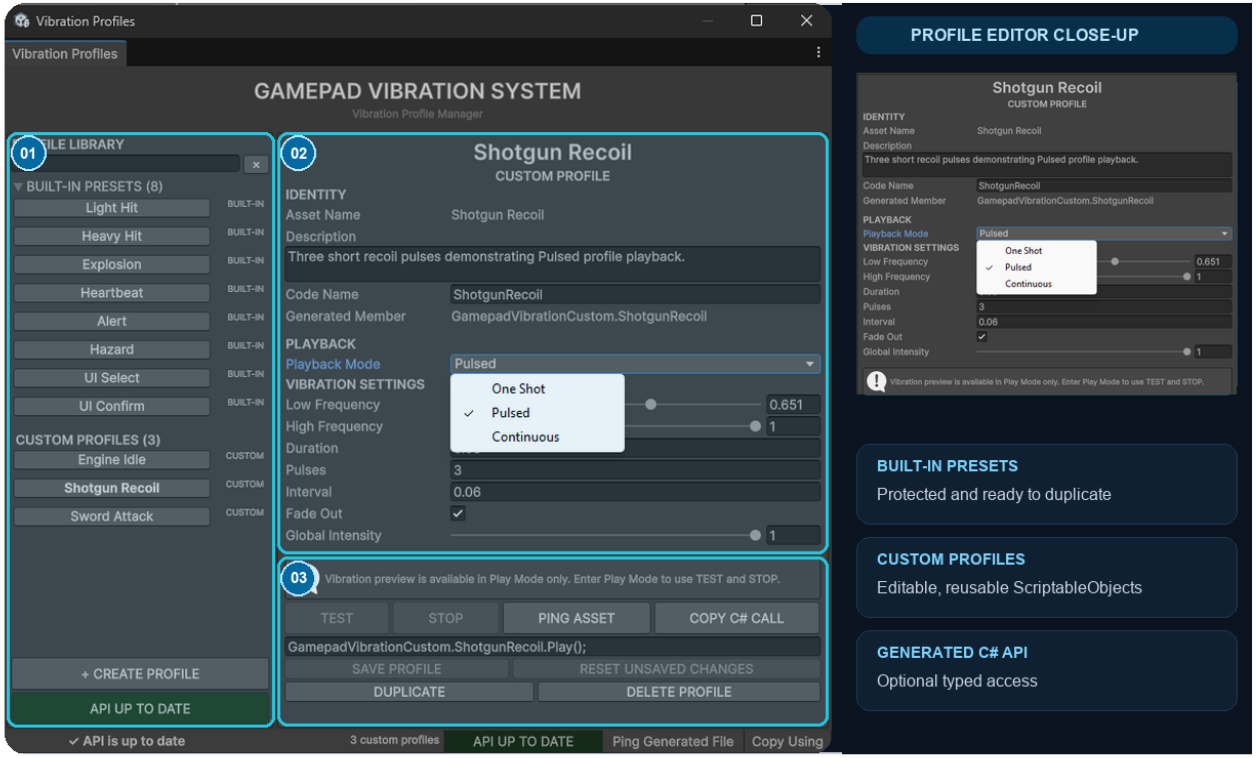
Flujo visual del Inspector: seleccionar la fuente, ajustar la respuesta y conectarla a eventos de gameplay.

PROFILE MANAGER

CREATE AND MANAGE VIBRATION PROFILES

Build reusable effects, preview them, and access them from components or C#.

GAMEPAD VIBRATION SYSTEM



PROFILE WORKFLOW

01 Organize profiles
Browse protected built-in presets and searchable custom profiles in one library.

02 Design the effect
Choose One Shot, Pulsed, or Continuous, then tune motors, timing, and Fade Out.

03 Save and use
Preview in Play Mode, duplicate profiles, ping the asset, or copy a ready C# call.

OPTIONAL GENERATED API

Profiles work directly as assets.
Code generation is never required.

CENTRAL PROFILE LIBRARY • BUILT-IN PRESETS AND CUSTOM PROFILES

UNITY INPUT SYSTEM

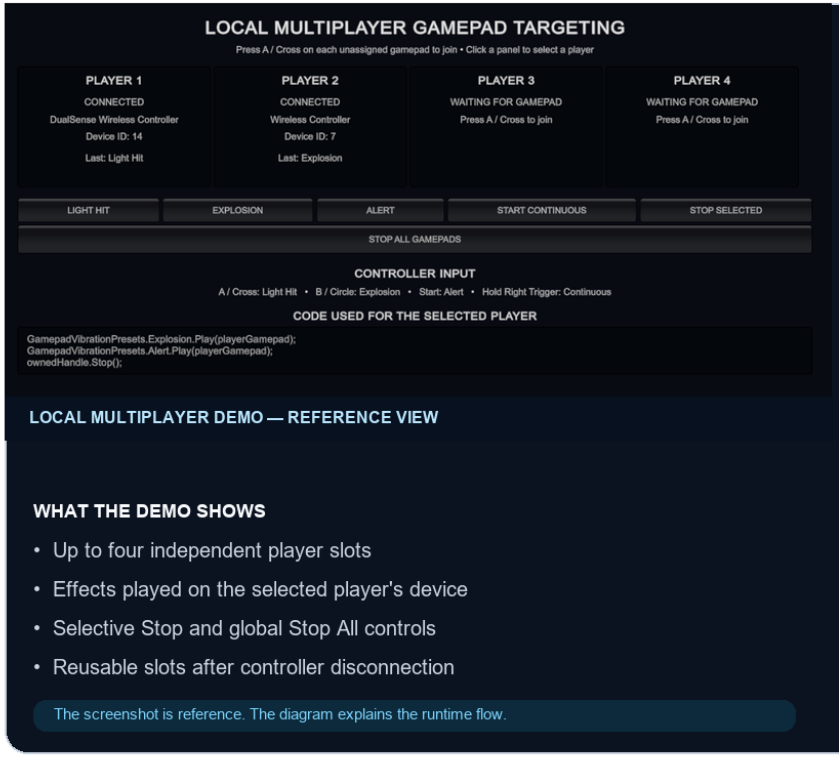
Profile Manager: organizar presets y perfiles custom, editar el efecto y acceder a acciones de uso.

LOCAL MULTIPLAYER

TARGET VIBRATION PER PLAYER

Each joined player keeps an independent gamepad target and vibration handle.

GAMEPAD VIBRATION SYSTEM

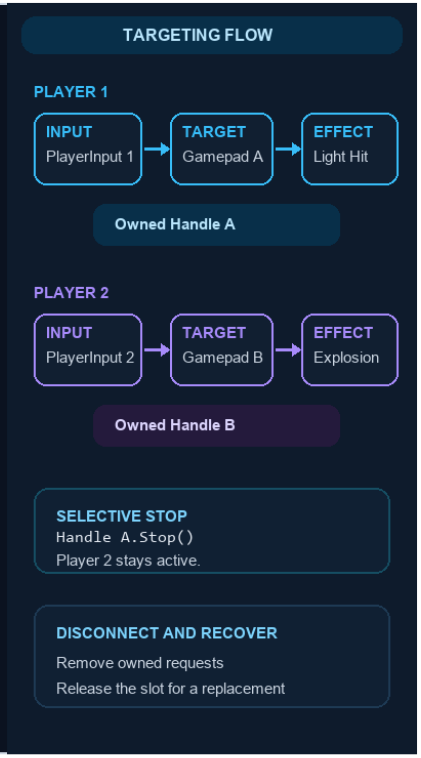


LOCAL MULTIPLAYER DEMO — REFERENCE VIEW

WHAT THE DEMO SHOWS

- Up to four independent player slots
- Effects played on the selected player's device
- Selective Stop and global Stop All controls
- Reusable slots after controller disconnection

The screenshot is reference. The diagram explains the runtime flow.



MULTIPLAYER FLOW

01 Join and bind
Each player joins with a specific gamepad. The project keeps that device reference.

02 Target the player
Pass the player's Gamepad or PlayerInput so rumble reaches only that device.

03 Stop independently
Each playback returns an owned handle. Stopping it does not affect other players.

NO INDEX-BASED TARGETING
Joined device references remain stable when other controllers connect or leave.

PER-PLAYER TARGETING • GAMEPAD AND PLAYERINPUT SUPPORT

UNITY INPUT SYSTEM

Multiplayer local: cada jugador conserva su gamepad, su efecto y su handle de parada independiente.